Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Технологий искусственного интеллекта

Дисциплина:

«Сетевые технологии»

ЛАБОРАТОРНАЯ РАБОТА № 4

«Трансляция адресов в ОС Linux»

Выполнил:

Бабич Александр Петрович, студент группы j3214

(подпись)

Проверил:

Трубников Алексей Васильевич

(отметка о выполнении)

(подпись)

Цель работы

Закрепить понимание принципов работы NAT и firewall, а также сформировать начальные навыки в конфигурировании NAT и Firewall на платформе Linux (CentOS 7).

Артефакты выполнения

1. Измененные параметры sshd из Части 2



Рис. 1: Настройки безопасности в /etc/ssh/sshd_config на хосте c7-2

В конфигурационный файл `/etc/ssh/sshd_config` на хосте c7-2 были внесены следующие изменения для повышения безопасности: запрет входа от root, лимит попыток ввода пароля (2), таймаут авторизации (30 секунд) и отключение UseDNS.

2 и 5. Итоговые файлы /etc/sysconfig/iptables с хостов (Часть 3 и 6)

Итоговый дамп правил `iptables` маршрутизатора c7-1, включающий политики по умолчанию, блокировки, NAT и проброс портов:

```
[ 1226.205325] device enp0s3 left promiscuous mode
[ 1302.735830] device enp0s3 entered promiscuous mode
[ 1308.877083] device enp0s3 left promiscuous mode
[ 1404.342162] device enp0s8 entered promiscuous mode
[ 1555.467603] device enp0s8 left promiscuous mode
[ 1556.788063] device enp0s3 entered promiscuous mode
[ 1556.905509] device enp0s3 left promiscuous mode
[ 1563.921892] device enp0s3 entered promiscuous mode
[ 1564.038194] device enp0s3 left promiscuous mode
[ 1679.505495] Warning: Unmaintained driver is detected: ip_tables
^C
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# cat /etc/syscomfig/iptables
cat: /etc/syscomfig/iptables: Нет такого файла или каталога
[root@localhost ~]# cat /etc/sysconfig/iptables
# Generated by iptables-save v1.8.10 (nf_tables) on Tue Sep 29 23:39:11 2026
*filter
:INPUT DROP [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
-A INPUT -s 14.12.0.0/18 -j DROP
-A INPUT -s 192.56.0.11/32 -j DROP
-A INPUT -i enp0s8 -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -i enp0s3 -p tcp -m tcp --dport 22 -j DROP
-A INPUT -p icmp -m icmp --icmp-type 0 -j ACCEPT
-A FORWARD -s 14.12.0.0/18 -j DROP
-A FORWARD -s 192.56.0.11/32 -j DROP
-A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -d 10.0.0.2/32 -p tcp -m tcp --dport 22 -j ACCEPT
-A FORWARD -s 10.0.0.0/24 -d 8.8.8.8/32 -p udp -m udp --dport 53 -j ACCEPT
-A FORWARD -s 10.0.0.0/24 -d 77.88.8.1/32 -p udp -m udp --dport 53 -j ACCEPT
-A FORWARD -s 10.0.0.0/24 -p tcp -m multiport --dports 110,80,443,8080,22 -j ACCEPT
-A FORWARD -s 10.0.0.0/24 -d 192.168.1.0/24 -p tcp -m tcp --dport 25 -j ACCEPT
-A FORWARD -s 10.0.0.0/24 -d 8.8.8.8/32 -p icmp -m icmp --icmp-type 8 -j ACCEPT
-A OUTPUT -p icmp -m icmp --icmp-type 8 -j ACCEPT
-A OUTPUT -p icmp -m icmp --icmp-type 0 -j DROP
COMMIT
# Completed on Tue Sep 29 23:39:11 2026
# Generated by iptables-save v1.8.10 (nf_tables) on Tue Sep 29 23:39:11 2026
*nat
:PREROUTING ACCEPT [8:1852]
:INPUT ACCEPT [7:1768]
:OUTPUT ACCEPT [1044:47452]
:POSTROUTING ACCEPT [1044:47452]
-A PREROUTING -i enp0s3 -p tcp -m tcp --dport 55022 -j DNAT --to-destination 10.0.0.2:22
-A POSTROUTING -s 10.0.0.0/24 -o enp0s3 -j MASQUERADE
COMMIT
# Completed on Tue Sep 29 23:39:11 2026
[root@localhost ~]# _
```

Рис. 2: Вывод cat /etc/sysconfig/iptables (хост c7-1)

```
root@d12:~# iptables-save
root@d12:~# iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
root@d12:~# W
```

Рис. 3: Базовые правила iptables на хосте c7-2

3. Команда и консольный вывод из Части 4 п.3 (nmap)

```
[root@localhost ~]# nmap 10.0.0.2
Starting Nmap 7.92 ( https://nmap.org ) at 2026-09-29 23:27 MSK
Nmap scan report for 10.0.0.2
Host is up (0.00077s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 08:00:27:C3:04:A5 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.02 seconds
```

Рис. 4: Сканирование портов с7-2 с машины с7-1 (открыты 22 и 80 порты)

4. Команды и существенные части консольного вывода Части 5

П. 1. Открытые соединения и ждущие сокеты на с7-2:

```
root@d12:~# ss -tuln
ss -tun
Netid    State     Recv-Q    Send-Q       Local Address:Port         Peer Address:Port    Process
udp      UNCONN    0         0                  0.0.0.0:68              0.0.0.0:*
udp      UNCONN    0         0                  0.0.0.0:68              0.0.0.0:*
tcp      LISTEN    0         128                0.0.0.0:22              0.0.0.0:*
tcp      LISTEN    0         1024               0.0.0.0:80              0.0.0.0:*
tcp      LISTEN    0         128                   [::]:22                 [::]:*
tcp      LISTEN    0         1024                  [::]:80                 [::]:*
Netid    State     Recv-Q    Send-Q       Local Address:Port         Peer Address:Port    Process
udp      ESTAB     0         1536          10.100.0.4:57396         192.168.0.1:53
tcp      ESTAB     0         0              10.0.3.15:22               10.0.3.2:60259
```

Рис. 5: Вывод команд ss -tuln и ss -tun

П. 4. Работа NAT (MASQUERADE): При перехвате трафика утилитой `tcpdump` на внешнем интерфейсе шлюза видно, что внутренний адрес узла с7-2 заменяется на адрес внешнего интерфейса маршрутизатора с7-1.

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
23:32:59.893432 IP 10.0.2.15 > 8.8.8.8: ICMP echo request, id 33871, seq 1, length 64
23:32:59.902778 IP 8.8.8.8 > 10.0.2.15: ICMP echo reply, id 33871, seq 1, length 64
23:33:00.893709 IP 10.0.2.15 > 8.8.8.8: ICMP echo request, id 33871, seq 2, length 64
23:33:00.903765 IP 8.8.8.8 > 10.0.2.15: ICMP echo reply, id 33871, seq 2, length 64
4 packets captured
8 packets received by filter
0 packets dropped by kernel
[root@localhost ~]#
```

Рис. 6: Перехват ICMP (ping) трафика после трансляции адресов

П. 6, 8. Тройное рукопожатие и флаги TCP (SYN, ACK):

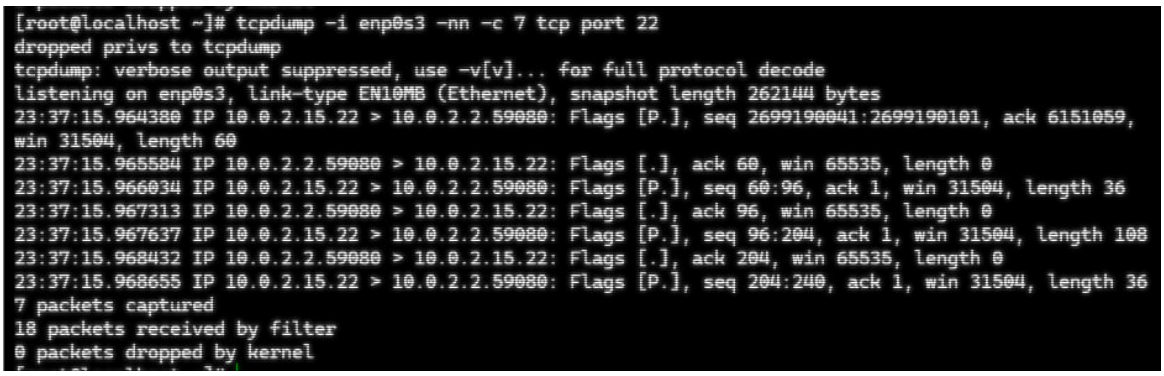


Рис. 7: Анализ флагов установки TCP-соединения через tcpdump (видны [S], [S.], [.], [P.])

6. Команда подключения из Части 7, п.1

Создание защищенного туннеля SSH для проброса веб-сервера Lighttpd на локальный порт основной операционной системы:

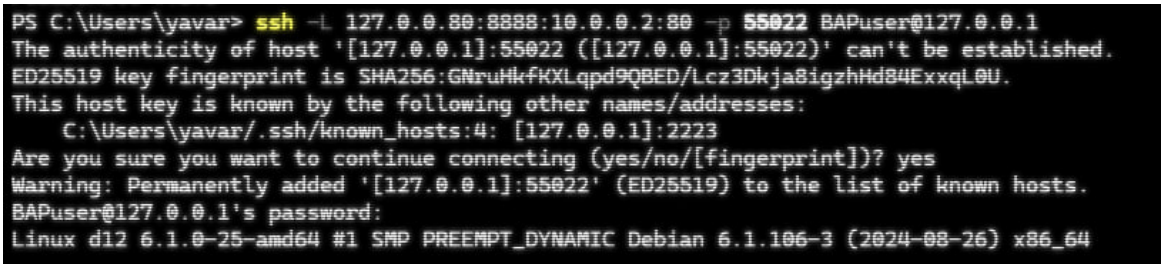


Рис. 8: Команда ssh -L 127.0.0.80:8888:10.0.0.2:80 -p 55022 bapuser@127.0.0.1

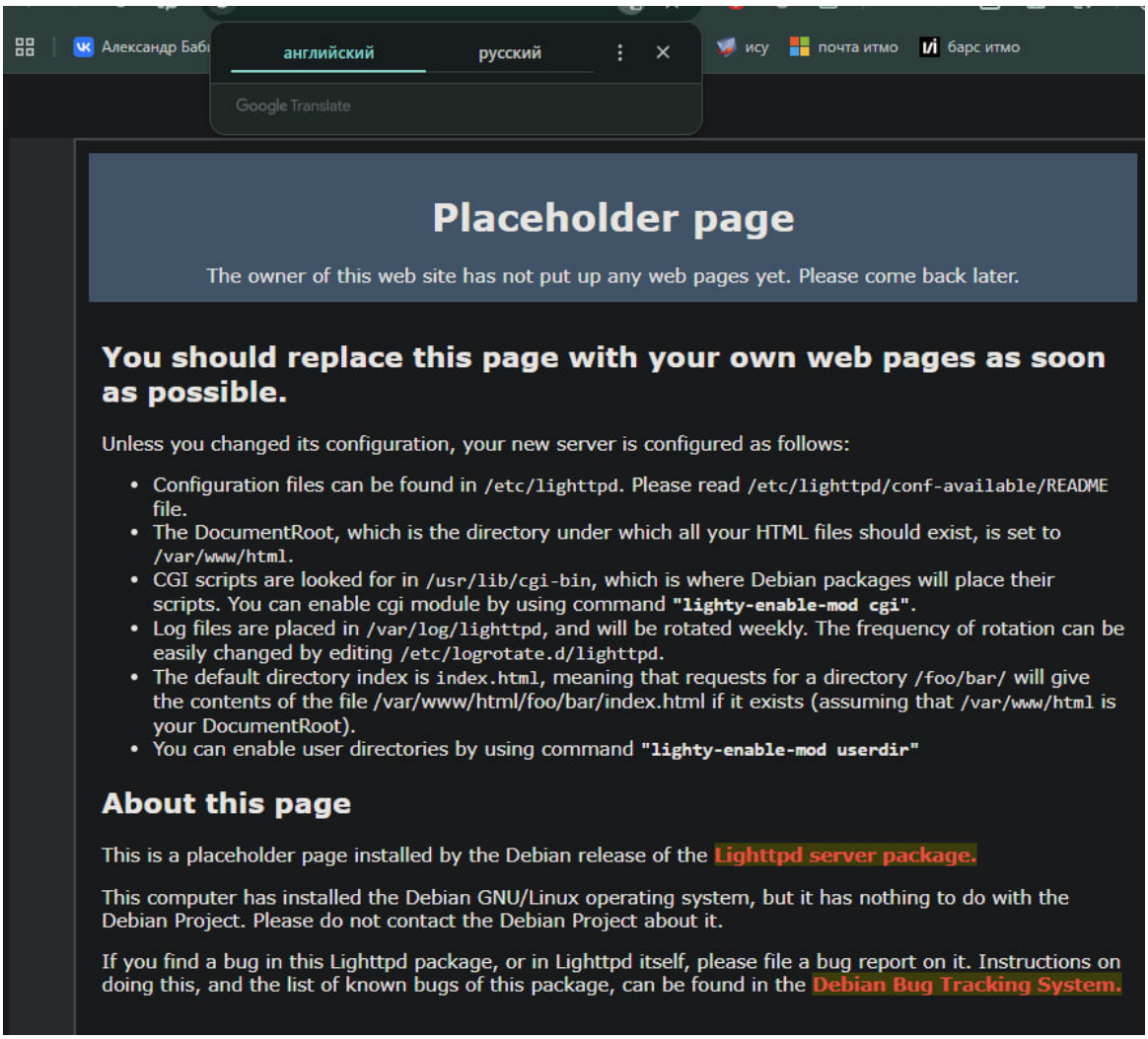


Рис. 9: Успешный доступ к странице веб-сервера через туннель

Ответы на вопросы и задания

1. **В чем разница между действиями SNAT или MASQUERADE? Когда уместно использовать одно, а когда другое?**
   Они делают одно и то же — подменяют внутренний IP-адрес на внешний. Разница лишь в том, что SNAT мы используем, когда у нашего роутера постоянный (статический) внешний IP-адрес. SNAT работает быстрее и меньше грузит железо. А вот MASQUERADE нужен, если провайдер постоянно меняет нам IP-адрес (например, выдает его динамически по DHCP). MASQUERADE каждый раз заново вычисляет нужный IP для отправки, поэтому он чуть тяжелее для системы, но зато сеть не падает при смене адреса.

2. **Какие цепочки и какие таблицы существуют в iptables по умолчанию?**
   По умолчанию в iptables есть несколько таблиц. Самые главные — это `filter` (используется для блокировки или разрешения пакетов) и `nat` (для подмены адресов и проброса портов). Ещё есть `mangle` и `raw` для хитрой модификации пакетов, но в обычных задачах они редко нужны.

   В таблице `filter` есть три стандартные цепочки: `INPUT` (для входящего трафика к самому серверу), `OUTPUT` (для исходящего от сервера) и `FORWARD` (для транзитного трафика, который идет сквозь наш шлюз). В таблице `nat` к ним добавляются цепочки `PREROUTING` (срабатывает до того, как система решит, куда отправить пакет) и `POSTROUTING` (срабатывает прямо перед отправкой пакета наружу).

3. **Как добавить новую цепочку? Как перенаправить в нее трафик?**
   Чтобы создать свою собственную цепочку, нужно написать команду `iptables -N ИМЯ_ЦЕПОЧКИ`. А чтобы направить в неё пакеты, мы добавляем правило в одну из стандартных цепочек с действием `-j ИМЯ_ЦЕПОЧКИ`. Например, вот так: `iptables -A INPUT -p tcp -j ИМЯ_ЦЕПОЧКИ`.

4. **Имеет ли смысл порядок правил?**
   Да, порядок очень важен. Файрвол читает правила строго сверху вниз. Как только пакет совпадает с каким-то правилом (например, ACCEPT или DROP), iptables сразу применяет это действие и перестает проверять список дальше. Поэтому разрешающие правила обычно ставят выше, чем общее правило блокировки всего остального. Если поставить `DROP` первым, то до разрешающих правил очередь просто не дойдет.

5. **Как с помощью iptables можно реализовать настройки, при которых брандмауэр пропускает пакеты тех соединений, которые были инициированы изнутри.**
   Для этого нужно использовать модуль `state` (или `conntrack`), который умеет запоминать, кто начал соединение. Мы добавляем разрешающее правило для состояний `ESTABLISHED` (уже установленные) и `RELATED` (связанные) в цепочки `INPUT` и `FORWARD` (команда выглядит как `iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT`). Благодаря этому файрвол будет жестко блокировать все попытки нового подключения снаружи (`NEW`), но спокойно пропускать обратно ответные пакеты для тех запросов, которые мы сами начали изнутри сети (например, когда мы пингуем внешний сервер).